02465 Project: Part 3

Tue Herlau
tuhe@dtu.dk

April 13, 2026

Formalities

- The deadline for this report is **Thursday 16th April, 2026** before 23:59.
- Submission of reports happen on DTU learn
- You can work in groups of 1, 2 or 3 students (but not 4)
- Collaboration policy: It is not allowed to collaborate with other groups on this project, except for discussing the text of the project with teachers and students enrolled on the course this semester. It is not allowed to communicate (or make available) solutions or parts of solutions to the project to other people. It is not allowed to use solutions from previous years, or solutions found on the internet or elsewhere.
- You can freely use code from the *exercises* when you solve the project, for instance the dynamical programming algorithm.
- Your overall evaluation will be based on your written answers and your UNITGRADE score. They will be weighted based on an assessment of the required work.

Preparing the hand-in:

Hand in these three files (please do not hand in a `.zip` file as this confuses DTU learn):

A `.tex` file with your written answers: Prepared this by modifying the template in `irlc/project3/Latex/02465project3_handin.tex` . Simply write your answers where it says **YOUR SOLUTION HERE**. I recommend keeping the layout as it is.

A `.pdf` file corresponding to this `.tex` file

A `.token` file containing your python-solutions: Generate this file by running the script `irlc/project3/project3_grade.py` . It is very important you do not modify this file.

In order to hand in the assignment on DTU Learn you must be part of a group. You can join a group on DTU learn from *My Course* → *Groups*.

Contribution table

DTUs exam rules require that each students contribution to the report is clearly specified. Therefore, for each element in the report, specify which student was responsible for it in the table in the template. **A report must contain this documentation to be accepted**. The responsibility assignment must be individualized. This means:

- For reports made by 3 students: Each section must have a student who is 40% or more responsible.
- For reports made by 2 students: Each section must have a student who is 60% or more responsible.

This is an external requirement. Ask me if you have any questions.

Code hand-in:

- Please keep the structure of the `irlc`-folder. All of your code which is specific to this report should be in the `irlc/project3/` directory. Solutions which use code outside the `irlc` folder cannot be verified and therefore cannot be evaluated.
- If you wish to use additional third-party libraries please discuss them with me first to ensure you are on the right track.
- Breaking or tampering with the UNITGRADE framework, for instance by reporting a false number of points or making your solution unverifiable, is potentially cheating. Code which is obfuscated to the point of being unreadable cannot be evaluated.
- This is not a programming course: Strange, long, undocumented, or downright disturbing solutions will be evaluated simply based on whether they work or not.

1 Jar-Jar at the battle of Naboo (`jarjar.py`)

In this problem, you have the ungrateful job of ordering Jar-Jar around during the battle of Naboo. Experience has taught you to keep things simple. You can either order Jar-Jar one step left one step right, and you want him to simply stay near a particular location $s = 0$ [1].

Specifically, the states Jar-Jar can be in are all integers:

$$\mathcal{S} = \mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}.$$

In each turn, you can tell Jar-Jar to move one step left ($a = -1$) or right ($a = 1$). Although Jar-Jar always does as you tell him, he will not stand still, and so you have to give him orders in each turn. If for instance Jar-Jar is in state $s = 10$, and you tell him to move left $a = -1$, he will transition to state $s + a = 9$, and in the next turn you need to give him a new order so he transitions to either $s = 8$ or back to $s = 10$.

[1] coincidentally, the landing spot of the trade federations landing vessel

The reward in each step is (minus) the distance to 0, i.e. $r = -|s|$. In the previous example, the reward you would obtain would be $r = -|10| = -10$. The environment does not terminate.

You are particularly concerned with the γ-discounted return ($0 < \gamma < 1$) you will obtain over time, and you want to be able to predict what it will be for any state s Jar-Jar may be in. The following exercises will highlight how we can both compute this approximately and exactly.

As a warm-up, you will implement the optimal policy for ordering Jar-Jar around. I.e., the policy which minimize the accumulated reward:

Problem 1 *Optimal policy*

Implement the optimal policy function π^* as the function `optimal_pi`.

Info:

No theory required: Simply think about what we want Jar-Jar to do.

Actions are encoded as $a = -1$ (left) and $a = 1$ (right)

There are in fact two optimal policies (why?). Either one is acceptable.

Next, let's consider the optimal value function:

Problem 2 *Simulating a finite approximation of the optimal action-value function*

Simulate an approximate version of the optimal action-value function $Q^*(s, a)$ in the state $s = 0$ and using action $a = 1$.

To do this, simulate the optimal policy for N steps, starting in $s_0 = 0$, to get $N + 1$ states $s_0, s_1, \ldots, s_N$ and the corresponding N rewards $r_1, r_2, \ldots, r_N$. Use these to approximate $Q^*(0, 1)$ for an arbitrary discount factor γ. Implement the result as `Q0_approximate`.

Info:

Think about what Jar-Jar will do under the optimal policy you just defined; it is something very simple and predictable.

The result should probably involve a loop.

Problem 3 *Analytically computing the optimal action-value function*

Next, we will analytically compute an expression for the optimal action-value function when computed on an infinite horizon, i.e. the same as the previous exercise but for $N \to \infty$. Your job is to derive a simple, closed-form expression for the optimal value function. You should derive the result for just two values: $Q^*(0,1)$ and $Q^*(1,-1)$.

The phrase *simple, closed form expression* means it should be expressed **without** using the sum-sign Σ.

Answer:

Using that ... we obtain

$$Q^*(0,1) = \cdots \tag{1}$$
$$Q^*(1,-1) = \cdots \tag{2}$$

therefore...

Info:

- The two values of (s,a) are not chosen at random: First use the Bellman optimality condition for $Q^*(0,1)$ to express $Q^*(0,1)$ in terms of $Q^*(1,-1)$.
- Then use the Bellman optimality condition for $Q^*(1,-1)$ to express $Q^*(1,-1)$ in terms of $Q^*(0,1)$.
- This should give you two equations with two unknowns.
- The phrase *simple, closed-form expression* refers to an expression such as $Q^*(0,1) = \gamma^3 - 1$ or $Q^*(0,1) = \frac{1}{\gamma}e^\gamma$ (neither of these are the correct solution, obviously).
- You can check your result by selecting a very large value of N in the previous exercise.

Problem 4 *Extend solution to all states and actions*

Extend the solution above, which works for two combinations of s and a, to get an *exact* method for computing $Q^*(s,a)$ for all values of s,a. The method can involve for-loops or recursions, but should not simply correspond to selecting a large value of N in problem 2. Implement this as `Q_exact`.

Info:

- You should extend your previous solution. Start by considering $Q(2, -1)$ and use Bellmans optimality equation to express this in terms of a known Q-value. Then consider $Q(3, -1)$ and get a good idea
- Similarly, for actions that point *in the wrong way*, use Bellmans optimality condition to express $Q^*(1, 1)$ in terms of a (now) known quantity
- Since the problem is symmetric around $s = 0$, the Q^*-functions also obey symmetries
- It is fine to use a (finite) for loop or recursion when computing $Q(s, 1)$ in general. However, your expression must be exact – i.e. do not use an infinite sum that you truncate etc.
- If you really want, you can avoid for loops and recursion completely by using a bit of math to obtain a closed-form expression for $Q^*(s, a)$. However, this is not required or really that advisable.
- Unitgrade will test values that I consider simple to obtain first, and then more complicated values.

Problem 5 *Modified reward function*

Suppose Jar-Jar can still be in all states, $\mathcal{S} = \mathbb{Z}$, and in each state there are still two actions $a = \{-1, 1\}$. However, assume that if Jar-Jar is in a state s then

- and takes action $a = 1$ then:
  - with probability p he will jump to state $s + 1$ (and get reward r_0)
  - with probability $(1 - p)$, he will jump to $s + 2$ (and get a reward of $2r_0$).
- and if he takes action $a = -1$ then:
  - with probability p he will jump to state $s - 1$ (and get reward r_0)
  - with probability $(1 - p)$, he will jump to $s - 2$ (and get reward $(2r_0)$

Given this information, and assume that $0 \leq \gamma < 1$, determine v_π as a function of γ and r_0.

Answer:

Using ... we get that ...:

$$v_\pi(s) = \dots . \tag{3}$$

Info:

- Does the policy matter?
- Solve a relevant Bellman equation
- Intuitively, before solving the problem, what can you say about the numerical value $v_\pi(s)$ for different values of s?

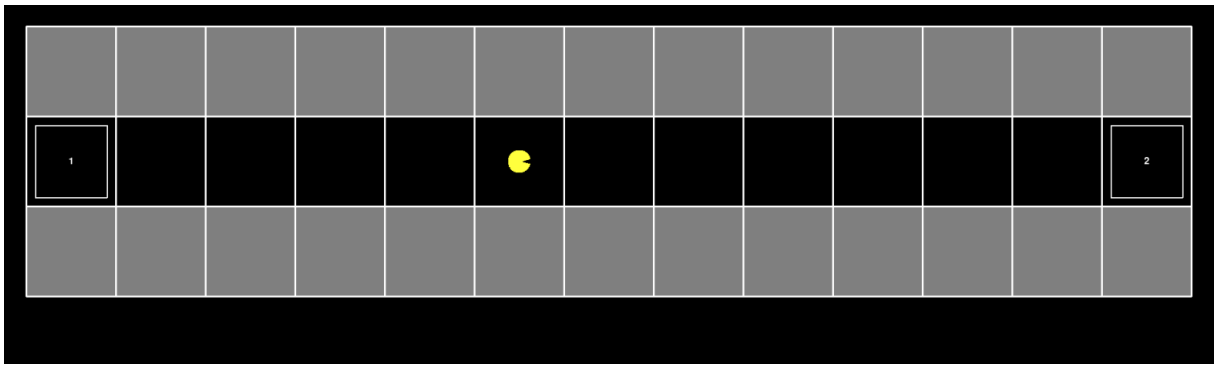

Figure 1: long bridge.

2 Optimal escape

In this problem, Han Solo is trying to escape from cloud city. This can be modelled as the standard grid-world problem using the familiar set of actions and states.

Concretely, we find yourself on a corridor illustrated in fig. 1. There are two doors you can exit through (left and right); the right-most has a reward of $R_T = 2$ and the left most a reward of $R_T = 1$. We assume there are n free tiles before the door on the left side and m on the right side. In the case shown in the illustration $n = 4$ and $m = 6$.

Problem 6 *Long bridge*

Assume the per-step living reward is $r_0 < 0$. Determine, as a function of γ, n, m, r_0, when it makes sense to go left, right, and when it does not make a difference, according to the optimal policy.

Simplify your answers so it does not contain the $\sum$-symbol or other similar constructions.

Answer:
Because ... we see that if $\cdots > \cdots$ one should go left, if $\cdots < \cdots$ one should go right, and when $\cdots = \cdots$ it makes no difference.

Info:

- First reason about what the optimal policy must do.

References